

SUMMARY

- Platform & Infrastructure Architecture:** Direct the technical strategy and architecture for backend systems and cloud infrastructure. Autonomously drive systems engineering: identifying foundational architectural risks, conducting R&D spikes, making core infrastructure decisions, and executing zero-downtime database and platform migrations on live production traffic.
- Infrastructure & Platform Governance:** Hold administrative and architectural responsibility across Google Cloud Platform (IAM policies, VPC topologies, and project lifecycles).
- Cloud FinOps & Infrastructure Economics:** Directly responsible for infrastructure capacity planning and cloud spend. Researched spend-based vs. resource-based CUD models and structured a 3-year Google Cloud Committed Use Discount (CUD) on compute instances that slashed long-term infrastructure costs.
- Cross-Team Technical Standards:** Define and maintain platform data contracts, binary IoT transport protocols, kernel-level traffic routing, and architectural boundaries—authoring the technical specifications that firmware, hardware, mobile, and backend teams implement against.

2026

Promoted to Senior Systems Architect | Declarative Cloud, Kernel Networking & Toolmaker

In April 2026, I was promoted to Senior Systems Architect. My mandate covers backend systems architecture, cloud infrastructure, kernel networking, and defining the platform data contracts and transport protocols implemented by connected devices and client apps.

Google Cloud Platform Administration & Infrastructure Operations

- Administrative & Platform Governance:** Sole administrator for Google Cloud Platform, managing foundational platform resources including IAM roles and policies, project topologies, VPC networks, firewall rules, and Cloud Storage lifecycles.
- Operational Independence & System Hardening:** Autonomously manage compute instances, automated backups, and storage buckets without external DevOps dependencies, ensuring high operational uptime and principle-of-least-privilege access across environments.

Feature Flags Engine & Controlled Rollouts (2026)

- The Problem:** Releasing high-impact changes across cloud APIs and mobile/firmware devices carried high blast radii if unexpected edge cases emerged in the field.
- The Solution:** Engineered an internal, lightweight feature flags system directly into the backend—enabling dynamic runtime evaluation, percentage-based rollouts, and instant kill-switches, allowing teams to dark-launch new capabilities without backend redeployments.

100% Declarative Infrastructure for Marketing Website (Non-Core): Terraform / OpenTofu (Cloud Run & Cloud SQL)

- The Scope & Isolation:** Non-core public marketing website. The company's public marketing website was previously hosted on manually configured virtual machines, lacking clean isolation from core platform resources and declarative reproducibility.
- The Solution:** Decoupled the marketing website completely from core backend infrastructure into its own isolated Google Cloud project driven 100% by Terraform / OpenTofu IaC: executed a zero-downtime DNS and workload cutover, migrating the marketing site onto containerized Google Cloud Run paired with managed Google Cloud SQL (PostgreSQL), codifying VPC networking, service accounts, IAM bindings, secrets, and database provisioning with zero manual cloud console mutations—ensuring marketing workloads have zero blast radius on core IoT backend systems.

Bare-Metal Homelab & Cloud-Native Engineering (Proxmox, k3s, Nomad, EKS)

- Exploring Future Platform Containerization:** Maintained an on-prem bare-metal host running Proxmox VE to evaluate container orchestration and workload isolation: automated VM provisioning via OpenTofu and `cloud-init`, spiked lightweight Kubernetes with k3s, evaluated HashiCorp Nomad, and spiked AWS EKS using Terraform.

Concurrency Overhaul: Dynamic IoT-Aware ThreadPool vs. ForkJoinPool

- The Bottleneck:** Default JVM concurrency models struggled with bursty IoT packet waves, either wasting memory on idle threads or inducing packet drops during sudden surges.
- The Implementation:** Replaced the legacy pool with a custom, load-adaptive dynamic ThreadPool that dynamically scales worker capacity in response to real-time incoming packet velocity and queue saturation. Eliminated legacy technical debt: obsolete dynamic TCP port provisioning flows and redundant in-memory state tracking.

Low-Level Kernel Networking & Edge Protocol Translation

- In-Kernel NAT64 & Dual-Stack Routing (Jool + nftables):** Addressed a critical network partition where cellular IoT clients operated exclusively over IPv6 while backend systems ran on IPv4. Ruled out userspace proxies (Nginx/HAProxy) that would drain burstable CPU credits; deployed the **Jool** Linux kernel module for stateful NAT64 (RFC 7915) alongside `nftables` DNAT, cleanly partitioning non-overlapping source port ranges on a shared public IPv4.
- Kernel-Level Packet Reflection & Dynamic Rate-Limiting:** Offloaded UDP NAT traversal echo responses directly into the Linux kernel using `nftables` packet reflection at prerouting priority (`notrack`). Implemented kernel-level DDoS/abuse protection using `nftables` dynamic sets (`limit rate over 10/second burst 20 packets at prerouting priority -301`) to drop traffic sweeps before socket allocation.
- Academic Preprint (Zenodo):** Authored and published *Connection-Agnostic Presence Tracking for Stateless Distributed Backends*: formulated a Redis-based architecture using sorted sets scored by expiration deadlines and throttled batch writes to track IoT device online/offline transitions across stateless backends with mathematically bounded detection latency.

Zero-Downtime Storage Architecture: Decoupling In-DB Payloads to Google Cloud Storage (GCS)

- The Architecture Challenge:** Legacy systems stored raw binary image payloads (Base64) directly inside primary MongoDB collections, degrading transactional throughput, ballooning nightly backup archives, and polluting WiredTiger cache memory.
- Multi-Phase Rollout & Dual-Write Strategy:** Conceived and orchestrated a zero-downtime, multi-stage migration lifecycle coordinating mobile clients and backend services: designed dual-write API contracts with document-level guardrail flags (`imageMigrated`) preventing legacy client overwrites, engineered an asynchronous backfill pipeline streaming decoded historical blobs to GCS with automatic stream MIME sniffing and immutable caching headers (`Cache-Control: public, max-age=31536000, immutable`), and handled corrupted legacy payloads with clean asset fallbacks.
- Graceful Cutover & Database Purge:** Maintained backward compatibility during a 2–3 day mobile client rollout buffer (~99% adoption), followed by a second deployment deprecating legacy Base64 endpoints and running MongoDB `$unset` operations to completely purge in-document blobs and migration flags.
- The Measurable Outcome:** Zero user downtime across live production. Slashed primary database storage by **54x** (3.6 GB → 67 MB, a 98% reduction in backup footprint), dropped runtime server memory footprint by **89%** (4.6 GB → ~500 MB), and eliminated binary blob overhead across all collections.

Production Database Administration (MongoDB): Schema Refactoring & Compound Index Optimization

- The Challenge (Sustained Heavy-Load Production):** As IoT write velocity and active homes surged, the primary production MongoDB cluster faced elevated disk I/O and memory pressure in the WiredTiger cache caused by legacy schema anti-patterns and accumulated index sprawl.
- Query Plan Audits & Index Hygiene:** Profiled live production query traffic using `explain` execution stats and slow query logs. Audited and pruned redundant, low-selectivity, and duplicate indexes that were imposing heavy write overhead on high-frequency IoT inserts and wasting working-set RAM.
- Compound Key Engineering & Schema Optimization:** Engineered selective compound indexes applying strict key ordering (Equality, Sort, Range — ESR pattern), ensuring high-frequency queries were satisfied entirely within index trees and eliminating expensive collection scans and in-memory sorts. Refactored legacy document schemas to curb unbounded growth and eliminate fragmented on-disk document allocations.

2025 Senior Software Engineer | Production Observability, Modern Infra & High-Performance Pipelines

In 2025, the research from late 2024 materialized into production infrastructure. I moved the company away from single-node disk-dependent operations toward dedicated observability, declarative infrastructure, and purpose-built ingestion pipelines.

Production Log Observability Rollout: Grafana & Loki (Retiring `hlogger`)

- The Operational Challenge:** Production applications dumped raw log files directly to VM disks, creating I/O pressure and requiring engineers to manually filter files or use custom tooling.
- The Implementation:** Provisioned an isolated observability VM and deployed Grafana and Loki for centralized log ingestion and querying. Managed service and scrape configurations in a git-tracked directory (pragmatic GitOps). Gracefully retired `hlogger` and disk log dumping, giving the engineering team real-time indexed search across live services.

High-Volume Telemetry Migration: ClickHouse, Delta+ZSTD Codecs & GCS Offloading

- Filesystem Inode Exhaustion:** The production VM faced recurring disk exhaustion from telemetry saved across deeply nested directory trees (`/aa/bb/cc/...`), causing severe filesystem inode depletion.
- Zero-Downtime Migration & Partition Backfilling:** Engineered a zero-downtime live migration pipeline: rather than running slow, memory-intensive `INSERT INTO` queries, backfilled and restored historical data at the partition level while live device writes continued uninterrupted. Devised full operational management with partition-optimized backup and disaster recovery restore strategies (archiving and restoring yearly partitions to/from GCS).
- Schema & Codec Engineering:** Modeled columnar ClickHouse schemas (`device_health` , `pal_2024` activity logs, and `crm_logs`) with tight data types: `FixedString(23/29)` , `LowCardinality(String)` , Delta encoding, and ZSTD compression. Slashed storage footprint by over 95% (compressing `device_health` telemetry from 17 GB down to 300 MB on disk) with composite primary keys optimizing block skip scans.

Staging Environment Infrastructure & Pre-Production Parity

- Pre-Production Validation Gate:** Replaced direct-to-production releases with a dedicated validation workflow, mitigating deployment risk and regressions.
- The Architecture:** Configured a dedicated on-prem server behind a static public IP replicating production topology. Established a strict deployment promotion gate where all features and refactors were verified on staging before receiving production tickets—dramatically reducing hotfixes.

Decoupled Data Architecture: Go + gRPC Device Health Service (Encapsulating ClickHouse)

- **Decoupled Architecture:** Decoupled ClickHouse from the core application monolith by building a standalone, lightweight Go microservice communicating over binary gRPC with strict Protocol Buffers contracts, deployed on an auxiliary VM.

SNode Architecture & Git-Driven Technical Specifications

- **Virtual Node Composite Abstraction:** Designed the architectural federation allowing disjoint physical hardware devices (e.g. multiple dimmers) to aggregate into a single composite Virtual Node (a subtype of SNode) that behaves as one unified device.
- **Establishing Technical Specifications:** Established a centralized, Git-based technical knowledge base: version-controlled specifications, binary protocol definitions, and API contracts that firmware, mobile, and backend teams review and align against prior to hardware and client releases.

Multi-Tier Rate Limiting & Pre-Production Telemetry Stack

- **Defense in Depth (Nginx + Bucket4j):** Configured reverse-proxy rate limiting in Nginx (HTTP 429) and token-bucket application rate limiting via Bucket4j inside the gRPC Activity Log service.
- **Staging Telemetry Stack (Docker Compose, VictoriaMetrics, Alloy):** Built a complete pre-production telemetry stack using Docker Compose: deployed Grafana, Loki, VictoriaMetrics, VictoriaLogs, and Alloy; evaluated and migrated away from Mimir; created and imported dashboards for host and container resource metrics.

Cloud-to-Cloud Integration: Yale Smart Locks (2025)

- Integrated Yale smart locks natively into the cloud platform: designed cloud-to-cloud OAuth integration, handled device state synchronization, access filtering, and real-time push notifications for remote lock and unlock events.

Technical Recruitment & Engineering Standards (Late 2025 – 2026)

- Designed a structured, multi-step hiring flow and evaluation rubric for backend engineering. Formulated practical technical assessments evaluating systems thinking and TDD, conducted interviews across stages, and successfully hired an engineer into the team.

2024

Promoted to Senior Software Engineer | Security Hardening, Deep Profiling & Resilience

In 2024, I expanded focus from operational stability to proactive systems engineering—profiling JVM runtime internals, closing security gaps with mutual TLS, and engineering resilient automated recovery mechanisms.

Zero-Trust Internal Tooling: `hlogger` (Go, mTLS & Custom PKI)

- **The Security Flaw:** Firmware engineers previously had raw SSH access to production Linux instances to read device logs—a severe audit and security vulnerability.
- **The Solution:** Built a secure client-server diagnostic tool in Go using Mutual TLS (mTLS): engineered a complete PKI hierarchy from scratch (Root CA, Intermediate CA, client certificates with CRL support). The server enforced mutual authentication, audited commands, and streamed filtered logs without exposing interactive shells.

Production Network Recovery & Deployment Hardening

- **Incident Triage:** Diagnosed and resolved a critical production network outage triggered by an interrupted `iptables` cutover state under high lag, rapidly restoring traffic routing.
- **Architectural Remediation:** Overhauled the deployment tooling to be strictly atomic and idempotent: implemented automated pre-flight health checks, atomic kernel routing swaps, and automated rollback guards to eliminate race conditions and partial state corruption during cutovers.

JVM Deep Profiling & Garbage Collection Engineering

- **Profiling Runtime Internals:** Instrumented the runtime with `async-profiler`, Java Mission Control (JMC), and VisualVM, generating flame graphs under live traffic. Mitigated allocation churn with pooled buffers and `ThreadLocal` allocations; pinned heap boundaries (`-Xms = -Xmx` at 4GB) and tuned GC for low pause times.
- **Hot-Path Zero-Allocation Formatting:** Identified CPU hotspots in binary packet decoding loops driven by `String.format`; refactored to Java 17 `HexFormat` and bitwise operations, cutting per-packet CPU overhead.
- **Redis Client Forensics:** Evaluated Redis drivers (Jedis vs. Lettuce) for async I/O; debugged silent "ghost connections" and poisoned connection pool states in Redis Pub/Sub that caused cascading application timeouts.

High-Throughput Telemetry R&D: ClickHouse Evaluation & Benchmarking

- Manually evaluated distributed logging and analytics engines (ELK vs. OpenSearch vs. Loki; Druid vs. Hadoop vs. ClickHouse). Identified ClickHouse as the optimal engine for high-throughput IoT time-series telemetry.
- Attended the inaugural Apache Kafka meetup in Bangalore (first official Kafka event in India) and Thoughtworks distributed systems events.

Device Provisioning Rewrite & Hardware Replacement Operations (Late 2024 – Early 2025)

- Extracted and overhauled provisioning logic from the monolith: implemented strict node validation assertions and completely rewrote the WiFi onboarding state machine. Built the replacement flow for failed hardware nodes, allowing hardware swaps without losing room mappings or automations.

Consumer-Driven Contract Testing Spike: Pact (2024–2025)

- Executed an exploratory R&D spike with Consumer-Driven Contract Testing (Pact) on a project to evaluate automated contract verification between frontend clients and backend APIs, analyzing workflows to catch contract breakages before deployment.

2023

Software Engineer | Production Reliability & Systems Engineering

In early 2023, I assumed full technical ownership of the backend platform and cloud infrastructure, prioritizing production stability, kernel and resource tuning, and automated incident recovery while driving feature roadmap delivery.

Linux Systems Forensics: File Descriptor Limits & I/O Starvation

- **Resource Exhaustion & Thread Failures:** Diagnosed recurring production stalls where system resource limits prevented thread allocation, locking out SSH access and impacting VM availability.
- **Diagnostics & Fix:** Used `sar`, `vmstat`, `iotop`, and `pidstat` to diagnose log flooding and `iowait` starvation. Discovered that shell `ulimit` and PAM were bypassed by `systemd`; reconfigured `systemd` unit limits (`LimitNOFILE=262144`), reloaded daemons, and tuned disk I/O scheduling using `nice` and `ionice`.

Database Connection Pool Forensics (MongoDB)

- Traced recurring database timeouts to an internal auxiliary service leaking unpooled MongoDB connections on requests; re-architected the service to use pooled connection lifecycles, immediately stabilizing database clusters.

Platform Expansion: Multi-Hub Automations & Ecosystem Integrations

- Extended the scene and rule automation engine to support multi-hub environments—handling nested scene fragments for large command payloads and multi-hub scene sync.
- Engineered and stabilized cloud-to-cloud voice integrations across Google Home and Amazon Alexa.

Critical Production State-Sync Remediation (Cloud & Hub Data Divergence)

- Discovered a severe, months-old bug in legacy sync code where flawed ID generation silently corrupted scene and rule automations on physical hubs. Rewrote the sync logic for deterministic conflict resolution and executed zero-downtime live production migration and state-reconciliation scripts across thousands of active homes without breaking customer automations or dropping hub connectivity.

Cloud FinOps & Infrastructure Economics: 3-Year Committed Use Discount (CUD)

- Evaluated cloud spend and compute efficiency models, analyzing spend-based versus resource-based commitments. Executed a 3-year resource-based Google Cloud Committed Use Discount (CUD) on core compute instances, substantially reducing long-term compute overhead.

Delivery Infrastructure: Bitbucket to GitHub Actions Migration

- Ported all build, test, and release pipelines from Bitbucket to GitHub Actions while preserving automated webhook zero-downtime cutovers without service interruptions.

Architecture Research & Extreme Programming (XP) Foundations

- Researched horizontal scalability patterns (Kafka vs. lightweight queues, Kubernetes trade-offs) and immersed in Extreme Programming (XP) and London-style TDD as a non-negotiable safety net.

2022

Software Engineer | Platform Modernization & Delivery Automation

Joined as primary backend engineer to modernize the IoT platform, introduce automated CI/CD delivery pipelines, upgrade legacy services, and establish production reliability standards.

Zero-Downtime Blue-Green Deploys & Automated Delivery

- **The Reality on the Ground:** Deployments were completely manual (SFTP'ing Jetty WAR files and restarting processes), causing recurring downtime.
- **Automated Delivery & Kernel Switching:** Engineered an automated CI/CD pipeline on Bitbucket with automated server webhooks. Cut over traffic at the Linux kernel level using `iptables` DNAT in both `PREROUTING` (external) and `OUTPUT` (loopback) chains, explicitly flushing the `conntrack` state table for immediate cut-over without dropping active requests.

IoT Hub Load Simulation & High-Concurrency Scaling (C10K)

- Tasked with simulating hundreds of concurrent IoT hubs replaying traffic logs. Encountered the C10K bottleneck where 1:1 thread-per-socket allocation exhausted JVM stacks; dove into Java NIO (`Selector`, `SocketChannel`), Go goroutines, and early Project Loom preview builds—fundamentally reshaping my mental model toward async and non-blocking I/O.

Spring 4 to Spring Boot 2.7.5 Migration: Unlocking TDD

- Untangled legacy XML and bean configurations to execute a major migration from Spring 4 (JDK 8) to Spring Boot 2.7.5 (JDK 17), cutting over live with zero downtime via the kernel blue-green pipeline. The primary driver was developer velocity: unlocking modern Spring test slices and establishing Test-Driven Development (TDD) across the backend.
- Apple HomeKit Integration Spike: Protocol Forensics & HAP Bridging
- Forked and adapted an open-source Java implementation of Apple's HomeKit Accessory Protocol (HAP), successfully engineering a backend bridge with mDNS/Bonjour discovery, cryptographic pairing (SRP and Curve25519), and session encryption. While the backend bridge succeeded, the initiative was shelved after client application integration stalled.

OPEN-SOURCE SYSTEMS & RESEARCH PROJECTS

stopgap Kotlin · Maven Central (‘2.8.0’)

Modern microservice framework built on **Helidon SE (Nima)** + **Project Loom** virtual threads. Features compile-time dependency injection via **KSP** (eliminating runtime reflection overhead) and an integrated three-tier testing harness (unit → in-process integration server → Docker E2E via Testcontainers).

assertgo Go · Generics

Type-safe testing assertion library built with modern Go generics. Provides a fluent API, chainable negation (`Not()`), custom matchers, and zero external dependencies.

relay Zig · Systems

Low-level TCP server implemented in pure Zig with a test-driven approach. Explores raw POSIX socket descriptors, manual memory management without libc runtime dependencies, port binding (`SO_REUSEADDR`), and preventing broken-pipe crashes (`SIGPIPE` suppression via `MSG_NOSIGNAL`).

hrh Rust · CLI (‘cargo install’)

Helm Release Helper — engineered during Kubernetes research to enable lean, declarative Helm releases without operator bloat. Reads YAML declarations and executes `helm upgrade --install` with diff previews and atomic rollback guarantees.

avoid Shell / Linux · OS

Minimal, purpose-built Linux distribution based on Void Linux for server recovery and lean headless appliances. Builds and publishes bootable `.img.gz` and `.qcow2` images via automated GitHub Actions pipelines.

c_oop C · TDD Spike

Object-Oriented Programming and London-style TDD in pure C. Implements struct polymorphism via function-pointer interface tables, heap-allocated lifecycle constructors, and isolated unit test harnesses.

TECHNICAL WRITING & SYSTEMS ENGINEERING ANALYSIS

Authored 20+ in-depth technical post-mortems and distributed systems essays published at [sku20.dev/blog](#):

Kernel Networking & Edge Routing

Betting on NAT64 Over a Proxy •Negotiating with Jool •Low-Level UDP Echo Server for NAT Traversal via nftables •Zero-Downtime Deployments with iptables

Concurrency & Runtime Internals

The 100ms Passport •Java Exceptions Swallowed: The ThreadPool Trap •ThreadLocal Optimizations and Project Loom •Optimizing Hex Formatting: String.format to Java 17 HexFormat

Distributed Systems & Database Reliability

Decoupling Blobs: A Zero-Downtime Migration from MongoDB to GCS •The Ghost Connection: Poisoned Redis Pub/Sub Connection Pools •Rate-Limiting: Flow Control vs. Quota Control •Hunting for a UDP Load Balancer

TECHNICAL COMPETENCIES & EDUCATION

LANGUAGES:	Kotlin, Java, Go, Rust, Zig, C, Shell, SQL
FRAMEWORKS & PROTOCOLS:	Spring Boot, Helidon SE (Nima), Project Loom, gRPC, Protocol Buffers, REST

CLOUD & INFRASTRUCTURE:	Linux (Debian, Ubuntu), Google Cloud (IAM, VPC, Cloud Run, Cloud SQL, GCS), AWS (Lambda, EKS spike), Terraform / OpenTofu, Docker, Proxmox VE, GitHub Actions
DATABASES & STORAGE:	ClickHouse, MongoDB, PostgreSQL, Redis
OBSERVABILITY & SRE:	Grafana, Loki, VictoriaMetrics, VictoriaLogs, Alloy, <code>async-profiler</code> , JMC, VisualVM, Flame Graphs, <code>sar</code> , <code>vmstat</code> , <code>iotop</code>
NETWORKING & SECURITY:	Socket programming (TCP/UDP), <code>iptables</code> , <code>nftables</code> , NAT64 (Jool), NAT traversal, mTLS & PKI (Root/Intermediate CA, CRL)
METHODOLOGY:	Extreme Programming (XP), London-Style TDD, Consumer-Driven Contracts (Pact spike), Testcontainers